DEEP RESEARCH REPORT AND BUILD PLAN

AURUM Decider

A Looped-Transformer Decision Language Model for Gold (XAUUSD) Trading — Designed, Trained, and Operated End-to-End on Kaggle

Prepared for: tskdev (Kaggle)
Research and Planning Phase — Version 1.0
November 6, 2026

Project AURUM · Research Deliverable · Choice-Order Invariant · Recurrent Depth

Table of Contents

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Executive Summary....1
2. Project Background and Objectives....2
    2.1 The Mandate....3
    2.2 Scope and Method of This Research....3
    2.3 Out of Scope....3
3. The Decision-Model Paradigm: Jev and Its Ecosystem....3
    3.1 From Generating Text to Making Decisions....3
    3.2 Jev and RLCD....4
    3.3 The Open Ecosystem....4
    3.4 Seven Engineering Lessons for Project AURUM....5
4. Looped Transformers: Research Deep Dive....6
    4.1 Why Depth Is the Thing Being Bought....6
    4.2 The Recurrent-Depth Architecture....6
    4.3 What the Evidence Actually Says....7
    4.4 The Thesis: Looping Fits Decision Models Better Than Generative Ones....7
    4.5 Risks and Our Adaptations....8
5. Base Model Selection....8
    5.1 Constraints....8
    5.2 Candidate Comparison....8
    5.3 Recommendation....9
6. Market Choice and Data Infrastructure: Gold (XAUUSD)....9
    6.1 Why Gold....9
    6.2 Data Source Evaluation....9
    6.3 Recommended Data Stack....10
    6.4 Features and Labels....10
    6.5 Augmentation and Counterfactual Pairs....10
7. Framing Trading as Decision Tasks....10
    7.1 State Serialization....10
    7.2 The Six-Task Decision Map....11

7.3 From Probabilities to Positions....11
7.4 What Stays in Code, Always....11
8. Kaggle as the Training and Automation Platform....12
8.1 Verified Capabilities....12
8.2 Quotas and Limits....12
8.3 Feasibility Math for a 0.45B Looped Trunk....13
8.4 The Automation Loop....13
8.5 Fallback Ladder....13
9. Backtesting and Evaluation Framework....13
9.1 Validation Methodology....14
9.2 Cost Model....14
9.3 Metrics and Gates....14
9.4 Mandatory Baselines....14
9.5 Statistical Robustness....15
9.6 Tooling....15
10. Proposed System Design: AURUM Decider....15
10.1 Architecture Overview....15
10.2 Scoring Head and Calibration....16
10.3 Parameter Budget....16
10.4 Training Recipe....16
10.5 Dataset Budget and Mixture....17
11. Implementation Roadmap....17
12. Risk Register and Mitigations....17
13. Conclusions and Next Steps....18
14. References....18

1. Executive Summary

This report presents the results of an end-to-end research phase for Project AURUM: a Jev-style, non-generative decision language model for trading gold (XAUUSD), whose transformer backbone is modified into a looped (recurrent-depth) transformer, trained and operated entirely on Kaggle using the account access verified during this research. The mandate for this phase was explicit: research everything end to end first, then plan, and only then build. Accordingly, this document contains no implementation; it contains findings, a system design, and a phased build plan.

The research covered six fronts: the decision-model paradigm introduced commercially by TypeSafe AI's Jev and its open-source ecosystem (JevBench, Kev, Laya, AnyJev, and the Bev Decider walkthrough that seeded this project); the looped-transformer literature from Universal Transformer (2018) through Huginn-3.5B and the 2026 survey work; base-model selection in the 0.4B-1.7B range; gold market data infrastructure; Kaggle's feasibility as a free training and automation platform, verified by a live API round-trip on the user's account; and backtesting methodology for financial machine learning.

The six headline recommendations that emerge from the research are as follows.

1. Adopt the Jev task interface - choice, noul (binary), and score questions over a serialized market state - as the single decision interface of the trading system, with hard risk limits and arithmetic enforced in code around the model, never inside it.
2. Build on Qwen3-0.6B truncated to its first 20 layers (approximately 0.45-0.47B parameters), the backbone validated by the Bev Decider walkthrough, because it uses standard scaled dot-product attention, which is the only kind compatible with the custom block-causal masks that guarantee choice-order invariance.
3. Implement looping exactly where the evidence supports it: a small recurrent core carved from the middle of the trunk with input re-injection, trained over a distribution of loop counts, giving a tunable think-harder dial at inference. For a single-pass decision model this carries none of the KV-cache and decoding-latency costs that make looping expensive in generative models.
4. Use HistData M1 data for XAUUSD as the primary training corpus (free, per-year archives, bid-ask consistent), Dukascopy as a cross-check, and Yahoo Finance for live daily updates during inference - all resampled to a multi-timeframe state (M15/H1/H4/D1).
5. Frame every trading decision as one of six typed tasks - direction, meta-gate, exit, stop distance, position size, and regime check - labeled procedurally with triple-

barrier and meta-labeling methods, which generates training data at zero marginal cost and with no leakage.

6. Train on Kaggle's free tier (30 GPU-hours per week, verified API automation loop), reserving rented A100 capacity as a fallback; every phase of the roadmap fits inside the free quota with 4-8 full training runs per week.

The overall verdict is positive and specific: the complete pipeline - data ingestion, decision-model training with a looped trunk, choice-order-invariance testing, backtesting with walk-forward validation, and a daily paper-trading harness - is feasible end-to-end on Kaggle at zero monetary cost, with a documented fallback ladder if compute needs outgrow the free quota. The largest risks are not computational but statistical: overfitting a sub-billion-parameter model to roughly two decades of gold history, and regime dependence of any learned edge. The design in Section 10 and the evaluation discipline in Section 9 are built specifically to keep those risks measurable and visible.

2. Project Background and Objectives

2.1 The Mandate

The project owner (Kaggle account tskdev) intends to build an AI decision model - not a chat assistant - that takes complete trading decisions on the foreign-exchange market, with gold (XAUUSD) as the primary instrument. The architectural direction is fixed by the owner: the model must follow the decision-model approach popularized by TypeSafe AI's Jev, and the transformer backbone must be modified into a looped transformer. The model must decide entries, exits, risk, and position size; portfolio-level decisions are a later extension. Everything must be backtested before any live consideration, and the owner explicitly requested a deep-research-first workflow: understand the best practical approach end to end, plan, and only then build.

2.2 Scope and Method of This Research

This report is the deliverable of that research phase. Its evidence base combines: (a) a live verification of the Kaggle API access on the owner's account, including pushing and executing a probe notebook on Kaggle infrastructure; (b) the owner's provided technical walkthrough of Bev Decider 0.4B, an independent 450M-parameter choice-order-invariant decision model; (c) primary sources on Jev and its ecosystem, including the TypeSafe AI launch material, the JevBench benchmark repository, and open-model cards for Kev and Laya; (d) the looped-transformer literature, anchored by the Huginn-3.5B recurrent-depth model, its underlying paper (arXiv 2502.05171), and 2026 survey and deep-dive analyses; and (e) established financial machine-learning methodology for labeling, validation, and

backtesting. Where a number in this report is an estimate or a planning assumption rather than a measured or published fact, it is explicitly marked as such.

2.3 Out of Scope

Three things are deliberately out of scope for this phase. First, live trading and broker integration: no execution infrastructure is designed here beyond a paper-trading harness. Second, real-money risk: all backtest and paper-trading results are treated as research artifacts, not as evidence of live profitability. Third, model building itself: no training code is written until the plan in Sections 10 and 11 is reviewed and approved by the owner.

3. The Decision-Model Paradigm: Jev and Its Ecosystem

3.1 From Generating Text to Making Decisions

Autoregressive language models make decisions the expensive way: they generate tokens sequentially, which introduces latency, non-determinism, and the possibility of hallucinated output that must then be parsed and validated. Decision language models invert this. They are non-generative, System 1 classifiers: the application supplies a state (any text or structured data), a question, and a bounded set of candidate answers; the model processes the whole packed sequence in a single parallel forward pass and outputs a calibrated probability distribution over exactly those candidates. Because the output space is architecturally bounded by the prompt, a decision model cannot invent an answer that was not offered, cannot emit malformed output, and runs in tens to hundreds of milliseconds rather than seconds.

The interface supports three task types, and they are sufficient to express the entire decision surface of a trading system. Choice tasks present one or more named options (which of these three actions should I take?). Noul (null) tasks present implicit yes/no candidates (should this signal be taken?). Score tasks present an ordered scale of levels and return both a distribution and a mean level index, mapping naturally onto ordinal quantities such as position size. Jev's own documented limits - weaknesses in arithmetic, date comparison, indirection, adversarial content, and structural consistency across separately asked questions - define the boundary of what belongs inside the model: semantic judgment under uncertainty, with exact computation kept in surrounding code.

3.2 Jev and RLCD

TypeSafe AI released Jev on 15 September 2026 as the first commercial System One model, trained with Reinforcement Learning for Calibrated Decisions (RLCD) - an algorithm that rewards reporting honest probabilities under strictly proper scoring rules, in contrast to RLHF's optimization for human preference. The published claims are substantial: on their

workflow benchmarks, roughly 194x faster and 445x cheaper than comparable LLM workflows, with 70-500 ms end-to-end latency, input pricing near $0.042 per million tokens, and a 0.00% structured-output error rate guaranteed by construction. Independent probing (notably Archer Hume's roughly 10,000-call reverse-engineering investigation, September 2026) and the open reproductions below suggest the core mechanism matches the packed-sequence, single-pass design described here. JevBench, the community benchmark (v1.4.2.2), scores systems as the mean of an Intelligence index - (accuracy minus chance) divided by (1 minus chance) - and a Calibration index based on expected calibration error, under a cost cap of $0.0646 per 1,000 decisions and an adjusted median-latency cap of 1.23 seconds. On that board, open models now cluster around and above Jev 1.13.0 itself (63.29), led by Imajev-4B at 67.37.

3.3 The Open Ecosystem

A small but fast-moving ecosystem of open decision models appeared within weeks of Jev's release. The table below summarizes the systems most relevant to this project, including the two that serve as direct architectural precedents.

Table 1: Open decision-model ecosystem relevant to Project AURUM

System	Backbone and size	Approach	Evidence most relevant to us
Jev 1.13.0 (TypeSafe AI)	Undisclosed (hosted)	Commercial System One model; RLCD; choice/noul/score API	Task interface, calibration emphasis, documented weakness list (arithmetic, dates, long irrelevant state)
Kev family (jaredpalmer)	Qwen3.5-Base 0.8B/4B/9B adapters; 27B full fine-tune	Adapters on frozen base; per-checkpoint temperature calibration; Jev-compatible API	Kev-0.8B reaches 0.697 accuracy on unseen sources (Jev: 0.857); Brier 0.416 - the sub-1B tier is viable but leaves headroom
Laya (Convai Innovations)	ModernBERT-large 421M bidirectional encoder; mmBERT 322M	Encoder + decision head scoring per-option tokens; ~33 ms; Apache 2.0; RLCD with proper scoring rules	Proves an encoder route exists (no causal-mask surgery); but flips ~40% of decisions under question rewording - a robustness warning for prompt design
AnyJev (Nokia Applied Research)	Any frozen LLM	Readout layers that turn a frozen model into a decision model	Fallback technique; zero backbone training

System	Backbone and size	Approach	Evidence most relevant to us
Bev Decider 0.4B (walkthrough)	Qwen3-0.6B truncated to 20 layers (~450M)	Joint encoder, LoRA on last 12 layers, self-attention scoring head, choice-order invariance by construction	The direct recipe for this project: 65.8% JevBench overall vs 61.4% for Kev-0.8B-class baselines; 100% tool-selection accuracy; total training cost $36
JevBench (community)	Benchmark harness	534 decisions; hard tier 30%; JSONL task format; public/held-out split	Our evaluation harness and invariance tests will follow its task format

3.4 Seven Engineering Lessons for Project AURUM

Read together, the ecosystem evidence yields seven concrete lessons that shape the design in Section 10.

1. Use a joint encoder, not a bi-encoder. Packing state, question, and all choices into one sequence lets cross-attention contextualize every candidate against the full market state; the cost is one forward pass instead of N+1.
2. Choice-order invariance must be built in, not hoped for. Resetting positional IDs at choice boundaries and applying a block-causal attention mask (choices see the state and themselves; the answer prompt sees everything) makes the output provably invariant to option order - a 10+ point accuracy factor in the Bev results.
3. Adapter-only training underperforms. Freezing the backbone and training only the scoring head lost more than 10 points of benchmark accuracy in the Bev walkthrough; LoRA injected into the final layers of the trunk was the minimum effective dose.
4. Calibration is a first-class deliverable. Every Kev checkpoint ships a fitted temperature; Jev's value proposition is calibrated confidence. For trading, confidence is not a display feature - it is the input to position sizing.
5. Sub-1B models are viable but leave headroom on hard reasoning. Kev-0.8B sits roughly 16 points below Jev on unseen sources. The looped transformer (Section 4) is precisely the mechanism for buying reasoning depth without buying parameters.
6. Keep arithmetic and hard rules in code. Every source agrees - Jev's own limits, JevBench's tier design, and the Bev result (arithmetic was its weakest category at 41.5%). The trading system computes risk, P&L, and position size in deterministic code; the model judges direction and conditions.

7. Prompt robustness must be tested, not assumed. Laya's 40% decision flips under rewording show that paraphrase stability is a real failure mode. Our evaluation suite will include a paraphrase-invariance test alongside choice-order invariance.

4. Looped Transformers: Research Deep Dive

4.1 Why Depth Is the Thing Being Bought

A fixed-depth transformer is a parallel circuit: every token passes through the same bounded chain of layers regardless of how hard the input is. Complexity theory makes the limitation precise - constant-depth, constant-precision transformers are confined to the complexity class AC0 (Li, Liu, Zhou and Ma, ICLR 2024), which excludes inherently serial problems. There are exactly two escapes: let the model emit intermediate tokens (chain-of-thought buys serial depth in token space, legibly), or let it re-run layers (recurrent depth buys the same serial depth in latent space, silently). Chain-of-thought costs context window, output tokens, and latency; recurrent depth costs only FLOPs. A looped transformer is not a smaller model - it is a model that thinks longer instead of being bigger.

4.2 The Recurrent-Depth Architecture

The modern reference design is Huginn-0.5B/3.5B (University of Maryland; arXiv 2502.05171), a 3.5B-parameter model trained on roughly 800B tokens with a three-part structure: a prelude P of ordinary layers that embeds the input; a recurrent core R of a few layers applied r times with one shared weight set; and a coda C that decodes the final state. Huginn is (2, 4, 2) - eight real layers - and at r=32 unfolds to 132 effective layers. Four implementation details are load-bearing, and every one of them is a published negative result if omitted.

1. Input re-injection on every iteration. The core computes s_i = R(e, s_(i-1)), where the embedded input e is concatenated with the running state and projected back down by an adapter matrix. Without injection, the iteration forgets the prompt: the fixed point becomes a property of the weights alone (demonstrated numerically: without injection, two different inputs converge to fixed points 1.8e-4 apart; with injection, 1.116 apart).
2. A random initial state, not the embedding. s_0 is drawn fresh from a truncated normal on every forward pass. Random initialization plus input injection produces path independence - different starts converge to the same trajectory - which is what makes it legitimate to change r at inference rather than querying the model out of distribution.

3. No step index. A step-conditioned core (iteration 7 knows it is iteration 7) was tried and reported to interact badly with path independence, producing models that cannot extrapolate across depth. The core must be blind to which iteration it is on.
4. Sandwich normalization. RMSNorm before and after each sub-layer. Plain pre-norm was one of the two configurations whose training runs collapsed at scale.

Training does not fix r. The loss is an expectation over r sampled per optimizer step from a heavy-tailed distribution (Huginn: log-normal Poisson with mean 32), and backpropagation is truncated to the last k=8 iterations, so activation memory depends on k, not on r. The result is a single checkpoint whose depth is an argument at call time: the same weights can be told to think harder about a hard input.

4.3 What the Evidence Actually Says

Table 2: Measured findings on looping, with implications for this project

Finding	Evidence	Implication for AURUM
Recurrences are worth less than unique layers, and the exchange rate is known	One recurrence is worth about r^0.46 in equivalent unique parameters (phi = 0.46, Huginn measurements)	A 4-layer core at r=3 adds roughly the reasoning depth of 9-10 fresh layers at zero parameter cost - meaningful for a 0.45B trunk
Looping buys reasoning, not knowledge	Three independent groups: extra passes recover 282% of the depth gap on math word problems vs 37% on closed-book QA	Trading decisions are judgment-under-pattern tasks, closer to the reasoning side; the knowledge (market facts) is supplied in the state text anyway
Returns flatten, and flatten faster for better-trained models	Gains saturate between 8 and 32 passes for lightly trained Huginn; one harder-trained model found a useful range of just 2 passes	Plan for small r (1-3). Even r=2 raises effective depth from 20 to 24 layers; treat larger r as an inference-time experiment, not a design assumption
Storage is saved; FLOPs and KV cache are not	At r=32 the KV cache is 10.6 GiB for one 4,096-token sequence vs 7 GB of weights (Huginn)	This cost largely evaporates for us: a decision model makes one forward pass and never decodes, so there is no growing KV cache at all (see 4.4)
Per-token adaptive depth is the frontier	Mixture-of-Recursions beats a 315M vanilla model using 167M parameters at equal FLOPs by routing each token to its own depth	A v2 direction: a light halting head could give easy regimes r=1 and ambiguous regimes r=3; deferred to Phase 6

Finding	Evidence	Implication for AURUM
2026 literature is consolidating	Survey and meta-analysis of looped language models (Azizi, 2026); Recurrent Looped Transformer (Zhang, 2026) splits a parallel encoder from a recurrent decoder; LT2 (arXiv 2605.20670) loops linear-attention hybrids; Jacobian analyses (arXiv 2609.01924) characterize stability	The design space is mature enough to copy a proven recipe rather than invent; our contribution is applying it to a decision model, which none of these cover

4.4 The Thesis: Looping Fits Decision Models Better Than Generative Ones

The two costs that make looping painful in production generative models simply do not exist in a Jev-style decision model. First, there is no KV cache: the model runs one forward pass over a fixed packed sequence and stops; the cache is allocated once, reused across all r iterations, and never grows. Second, there is no decode loop: latency is a single padded forward pass times r, so even r=3 on a 0.45B trunk stays far inside the 70-500 ms envelope that Jev itself treats as acceptable - and far inside the several-second horizon over which a trading decision on H1 bars must be made. Third, the invariance machinery of Section 3 (position-ID resets, block-causal mask) is computed once from the fixed sequence layout and reused identically on every iteration: looping changes only the hidden states, never the mask or positions. The result is that we get the reasoning-depth benefit of looping at close to its theoretical minimum cost. To our knowledge, no published system combines recurrent depth with a decision-model head; this is the novel component of Project AURUM, and it is novel in a low-risk direction - the unlooped model is exactly the r=1 special case.

4.5 Risks and Our Adaptations

Three risks follow from the literature, each with a planned mitigation. Training instability: sandwich norms and depth-scaled initialization are the published fixes, and because we fine-tune from a pretrained trunk rather than pretrain from scratch, we additionally initialize the injection adapter near identity so that r=1 reduces exactly to the unlooped baseline - the loop can then only help or be neutral, not catastrophically break the pretrained representations. Diminishing returns: the plan trains over r sampled from {1, 2, 3} and evaluates r in {1, 2, 3, 4} at inference, so the data decide the operating point rather than enthusiasm. Path independence is a from-scratch-pretraining property; in a fine-tuned trunk we do not rely on it - we simply verify empirically that inference at each r in range stays calibrated, and we always keep the r=1 checkpoint as the deployable fallback.

5. Base Model Selection

5.1 Constraints

The backbone must satisfy five hard constraints. (1) Standard full attention with SDPA-compatible implementations: the choice-order-invariance mechanism injects a custom 4D additive float mask and custom position IDs, which is only clean in models where every layer is conventional attention; hybrid architectures that interleave linear-attention or sliding-window layers (Gated DeltaNet, GDN) break the mask semantics. (2) Sub-1B parameters: Kaggle T4 memory and throughput, and the Bev evidence that a well-trained 0.45B decision model competes, make this the sweet spot. (3) Permissive license (Apache 2.0 preferred) for eventual open release. (4) Strong pretrained representations, since we fine-tune rather than pretrain. (5) A tokenizer with good numeric handling, because serialized market states are number-dense.

5.2 Candidate Comparison

Table 3: Backbone candidates for the AURUM trunk

Model	Size / layers	Attention	License	Verdict
Qwen3-0.6B	~0.6B / 28 (h=1024, GQA 16Q/8KV, 32k ctx)	Standard full attention (SDPA)	Apache 2.0	SELECTED - proven by the Bev recipe; truncating to 20 layers gives ~0.45-0.47B; ties embeddings keep vocab costs at 155M
Qwen3-1.7B	~1.7B / 28 (h=2048)	Standard full attention (SDPA)	Apache 2.0	Stretch option for a later accuracy push; roughly 2.5-3x the T4 training cost per run
Qwen2.5-0.5B	~0.5B / 24	Standard full attention	Apache 2.0	Viable fallback; older generation, weaker reasoning; no advantage over Qwen3-0.6B
Llama-3.2-1B	~1.2B / 16 (GQA)	Standard full attention	Llama Community License	Rejected - license conditions complicate open release; no architectural advantage

Model	Size / layers	Attention	License	Verdict
ModernBERT-large (Laya route)	421M / 24 bidirectional encoder	Bidirectional full attention	Apache 2.0	Attractive fast path (no causal-mask surgery; option-token scoring) but 512-1024 token context is tight for multi-timeframe states and world knowledge is weaker; kept as a v2 baseline for latency comparison
Qwen3.5 family (Kev route)	0.8B-27B	Hybrid GDN:full-attention ~3:1	Apache 2.0	Rejected for v1 - GDN layers break custom 4D mask semantics; Kev makes it work via adapters, but invariance by construction (our requirement) is cleaner on Qwen3

5.3 Recommendation

Qwen3-0.6B, truncated to its first 20 layers, is the recommended trunk. The truncation follows the Bev finding that the final layers of an autoregressive model specialize for next-token vocabulary prediction while core semantic representations live in the middle layers; dropping layers 21-28 removed vocabulary-head specialization without hurting decision accuracy, and left roughly 450M parameters. The discarded language-modeling head (2048-way projection to the 151,936 vocabulary) is replaced by the decision scoring head of Section 10. LoRA adapters on the last 12 trunk layers provide the trainable capacity, per the Bev ablation showing adapter-only training loses more than 10 points. Should Phase 3 show the looped 0.45B model plateauing below its evaluation gates, the same code path upgrades to Qwen3-1.7B within the same weekly Kaggle quota at roughly 2.5-3x wall-clock per run.

6. Market Choice and Data Infrastructure: Gold (XAUUSD)

6.1 Why Gold

The owner's prior choice of gold is well supported. XAUUSD is among the most liquid instruments in the world, trades nearly continuously (23 hours, five days a week), exhibits persistent volatility with exploitable session structure (Asian range compression, London

and New York expansion), and is driven by a compact, well-studied macro set - real yields, the dollar index, central-bank buying, and geopolitical risk - all of which can be summarized in the model's state text. A substantial recent literature (transformer-based gold futures prediction, 2025; LSTM/MLP with metaheuristic optimization, 2025; FinBERT-sentiment-augmented LSTM frameworks, 2025) confirms both feasibility and active interest. The honest counterpoints are equally clear: gold's behavior is regime-dependent (the 2022 bear and the 2024-2026 bull are different markets), top-tier news events (FOMC, NFP, CPI) produce gaps that invalidate any bar-based model for minutes at a time, and two decades of history limits the independent-sample count more than tick counts suggest. The design answers these with multi-timeframe states, regime features, event-time guards in code, and the evaluation discipline of Section 9.

6.2 Data Source Evaluation

Table 4: XAUUSD historical data sources evaluated

Source	What it offers	Limits	Role in AURUM
HistData.com	Free M1 bars and tick data for XAUUSD (plus XAGUSD, WTI, indices; 66 instruments), delivered as per-year ASCII archives, multi-year depth (early 2000s onward)	Manual per-year downloads; bid-side generic format; no warranty	PRIMARY training corpus - free, deterministic, bid-ask consistent, resample to M15/H1/H4/D1
Dukascopy	Free historical data feed built from its own tick data, CSV export from tick to monthly granularity; XAUUSD spot	Roughly 200,000 bars per export (chunked downloads needed)	Cross-check and gap-fill for the primary corpus; source for higher-fidelity spread studies
Yahoo Finance	GC=F COMEX gold futures front month, GLD ETF proxy; easy programmatic access	Front-month rolls create artificial gaps; rate limiting observed in our probe (HTTP 429); spot series unreliable	Live daily refresh in the inference/paper-trading kernels only
stooq	Free end-of-day XAUUSD with long history	Daily granularity only	Sanity check on daily bars
TwelveData / AlphaVantage	Vendor APIs with metals coverage	Free tiers are heavily rate-limited (hundreds of requests/day)	Not used in v1

Source	What it offers	Limits	Role in AURUM
Kaggle Datasets	Existing community gold datasets	Provenance and quality vary; often short histories	Not used for training; our own processed corpus will be published as a versioned private dataset

6.3 Recommended Data Stack

The pipeline ingests HistData per-year M1 archives for XAUUSD, validates them against Dukascopy on sampled windows (tolerance-based bar comparison), resamples to M15/H1/H4/D1 with proper session boundaries, computes the feature set below, applies triple-barrier labeling, and publishes the result as a versioned Kaggle dataset (Parquet, well inside the 20 GB limit at a few hundred MB). Yahoo Finance refreshes the most recent bars daily inside the inference kernel, with HistData monthly archives as the periodic re-anchor to correct any live-source drift.

6.4 Features and Labels

Per-timeframe features (computed on M15, H1, H4, D1): returns over 1/4/12 bars; ATR(14) as percentage; RSI(14); MACD histogram; distances of price to EMA20/EMA50/EMA200 in ATR units; Bollinger position; realized volatility (20-bar); swing structure summary (higher-highs/lower-lows flags); session tag (Asia/London/NY overlap); day-of-week; and D1 regime tags (trend vs range via ADX, volatility quartile). All features are point-in-time: computed from bars strictly before the decision bar, shifted to eliminate look-ahead. Labels come from the triple-barrier method (Lopez de Prado, Advances in Financial Machine Learning, 2018): for each decision point, an upper barrier at +m x ATR, a lower barrier at -n x ATR, and a time barrier of H bars; the label records which barrier was touched first. This converts noisy price paths into well-defined, horizon-consistent outcome labels, and it is fully procedural - Type 3 data in the Bev taxonomy, zero marginal cost, no LLM in the loop. Exit-action labels are the hindsight-optimal action (hold, trail, take-half, close) over the next K bars given the open position; size labels follow a volatility-targeting rule (risk fraction inversely proportional to realized volatility, capped); meta-gate labels ask whether a given rule-based signal, taken, would have hit the profit barrier before the stop - the meta-labeling pattern.

6.5 Augmentation and Counterfactual Pairs

The Bev walkthrough's most transferable dataset trick is the counterfactual pair: two states differing in one controlled detail, with the correct label flipping. The market analog is feature perturbation with recomputed labels - for example, the same state with the D1 trend tag flipped from up to down, or session moved from London to Asia, or volatility quartile shifted - with the triple-barrier label recomputed on the perturbed path. Template-based paraphrasing of the state serialization (multiple phrasings of the same numbers) attacks the Laya-style rewording fragility directly. Both are cheap, procedural, and multiply the effective dataset; the target corpus is roughly 120,000 training examples across the six task types, balanced so that no-trade and no-action classes do not swamp their alternatives.

7. Framing Trading as Decision Tasks

7.1 State Serialization

Every decision consumes one serialized market state: a compact, structured text block of roughly 400-600 tokens carrying the multi-timeframe feature set, current position information (side, entry, unrealized R, bars held), and the pending question. A representative fragment: D1: trend=up (ADX 31), vol quartile=3/4, price 1.8 ATR above EMA50; H4: RSI 63, MACD hist positive 5 bars, last swing HH at 2612; H1: ATR 6.2 (0.24%), EMA20>EMA50, distance to EMA20 = 0.9 ATR; M15: 3 consecutive up bars, session=London-NY overlap, day=Wed; position: flat; question: direction over the next 24 hours? Numbers are rounded to stable precision so token patterns repeat; the format is fixed and versioned, because Laya's rewording result warns that inconsistent phrasing is itself a distribution shift.

7.2 The Six-Task Decision Map

Table 5: Trading decisions mapped to Jev-style task types

#	Task	Type	Candidates / levels	Label source	Role in the policy
T1	Direction	Choice	Long / Short / No trade	Triple-barrier (sign of first barrier hit, or no trade if neither within H bars)	Entry decision
T2	Meta-gate	Noul	Take this signal / pass	Meta-labeling: would the rule-based signal have profited	Confidence gate that filters T1 signals
T3	Exit management	Choice	Hold / Trail stop / Take half / Close all	Hindsight-optimal action over next K bars given the open position	Manages open trades bar by bar

#	Task	Type	Candidates / levels	Label source	Role in the policy
T4	Stop distance	Choice	1.0 / 1.5 / 2.0 / 2.5 x ATR	Hindsight barrier analysis: widest stop not hit before profit barrier	Sets initial risk anchor for R accounting
T5	Position size	Score	Risk-per-trade levels 0.25 / 0.5 / 0.75 / 1.0 / 1.5 / 2.0 (%)	Volatility-targeting and drawdown-state labels	Converts confidence and regime into exposure
T6	Regime check	Noul	Volatility elevated? / Trend intact?	Computed regime features (procedural ground truth)	Cheap sanity gates guarding T1-T5

Two properties of this map matter. First, it is complete: entry, exit, stop, size, and gating - the owner's full decision surface - are covered, with portfolio allocation arriving later as a choice task over instruments once multi-asset data lands (Phase 6). Second, it is composable in code: the policy engine queries T6, then T1, gates with T2, anchors risk with T4, sizes with T5, and manages with T3 - every step typed, bounded, and logged with its probability.

7.3 From Probabilities to Positions

The calibrated probability output is the bridge to risk management. The base position size is fractional Kelly with a hard cap: risk fraction = min(cap, k x p x (b - (1 - p)) / b) using the model's calibrated win probability p and the reward-risk ratio b implied by the T4 stop and the profit barrier. The cap and the k multiplier are code-level constants the owner sets; the model never sizes positions by itself. This is exactly the division of labor the Jev ecosystem converged on - the model supplies honest uncertainty, code supplies arithmetic and hard limits.

7.4 What Stays in Code, Always

Position-size arithmetic, R-multiple accounting, max daily loss and max concurrent exposure rules, event-time guards (no new entries in the 30 minutes around top-tier scheduled news; flat over weekends unless the owner opts in), spread-aware entry filters, and order construction. None of these are model tasks; they are the type-safe wrapper that makes a probabilistic component safe to operate. The paper-trading harness of Phase 5 exists precisely to exercise this wrapper against live data before any human relies on it.

8. Kaggle as the Training and Automation Platform

8.1 Verified Capabilities

During this research phase, the complete Kaggle API round-trip was verified live on the owner's account (tskdev): authentication with the provided access token; listing and pulling the two existing notebooks (both empty starter kernels); pushing a new private notebook with internet enabled; executing it on Kaggle infrastructure (status polled to COMPLETE in roughly one minute); and retrieving its output log programmatically. The probe also fingerprinted the Kaggle Python environment, which matters for dependency planning. No blocking issues were found; the only operational frictions were the absence of a pre-installed Kaggle CLI in the orchestration sandbox (trivially installed) and HTTP 429 rate limiting from Yahoo Finance inside the kernel (handled with retries and headers in the design).

Table 6: Kaggle environment probe results (verified November 6, 2026)

Probe	Result
API authentication, kernels list/pull/push/status/output	All functional end-to-end
Python runtime	3.13.18
Pre-installed ML stack	PyTorch 2.11.0 (CPU build in probe; GPU builds available on GPU kernels), pandas 2.3.3, NumPy 2.1.3, scikit-learn 1.6.1, XGBoost 3.4.1
Outbound internet in kernels	Enabled and reachable (Yahoo responded; rate-limited with HTTP 429)
Existing notebooks on the account	Two empty starter notebooks - the project starts from a clean slate

8.2 Quotas and Limits

Table 7: Kaggle free-tier limits relevant to training operations

Resource	Limit	Planning consequence
GPU hours	About 30 hours/week (floor; Kaggle states it may be higher depending on demand); weekly reset	4-8 full AURUM training runs per week, plus ablations - the loop over r in {1,2,3} fits comfortably
Session length	12 hours maximum per interactive or committed run	Every training kernel must checkpoint and be resumable; runs are chunked at 8-10 hours with warm restarts

Resource	Limit	Planning consequence
Concurrent GPU sessions	1 interactive + 2 committed (background) GPU kernels	Two training runs can execute in parallel from the API - the automation loop exploits this
Working directory	20 GB persisted via Save Version	LoRA adapters (~100-200 MB each) plus tokenizer and config fit with wide margin; raw M1 archives (a few GB) live in attached datasets instead
Datasets	20 GB per dataset; private quota 20 GB total (public is free)	The processed feature/label corpus (a few hundred MB Parquet) and model-checkpoint datasets both fit privately
GPU types	NVIDIA T4 x2 or P100 (16 GB each)	Single-T4 LoRA training is the default; 2x T4 DDP is the reserve for the largest runs

8.3 Feasibility Math for a 0.45B Looped Trunk

Memory: the truncated trunk is roughly 470M parameters (about 314M in 20 non-embedding layers plus 155M tied embeddings); fp16 weights occupy about 0.95 GB. Trainable parameters - LoRA (rank 16) on the q/k/v/o and MLP projections of the last 12 layers, the injection adapter, and the scoring head - total roughly 12-15M (estimate); their AdamW states and fp32 master copies add well under 0.5 GB. With gradient checkpointing, activation memory at batch 8-16 and sequence lengths of 512-768 stays in the low single-digit GB. Total peak usage is approximately 3-5 GB against 16 GB on one T4 - comfortable headroom, and the reason 2x T4 is a reserve rather than a requirement. Throughput: at roughly 2 GFLOP per token for forward-plus-backward on a 0.35B-non-embedding model, a 120,000-example corpus at 2 epochs and average packed length near 600 tokens is of the order of 15,000-30,000 optimizer steps; on a single T4 at realistic efficiency this lands at roughly 3-7 GPU-hours per full run (estimate, to be calibrated in Phase 2). The weekly 30-hour budget therefore supports multiple full runs plus the smaller ablations that Phases 2-3 require.

8.4 The Automation Loop

The Bev walkthrough ran its training on a rented A100 with an autonomous agent monitoring checkpoints over SSH. Project AURUM replicates that loop with zero infrastructure cost, because the Kaggle API already exposes every primitive the agent used: the orchestrator (this assistant, on the owner's Kaggle token) generates a training notebook and pushes it; polls kernel status; pulls logs, metrics, and checkpoint artifacts from kernel output; analyzes results; and pushes the next iteration - hyperparameters, data-mixture updates, or code fixes -

as a new kernel version. Checkpoint resume works through versioned Kaggle datasets: each training kernel saves its latest LoRA adapter and optimizer state to /working as output, and the orchestrator attaches the prior version as an input dataset for the next chunk. Data updates flow the same way, as versioned private datasets. Everything is scriptable today; nothing in this loop is speculative.

8.5 Fallback Ladder

If compute needs exceed the free tier (for example, a Qwen3-1.7B trunk or a large r sweep), the ladder is: (1) shift all CPU-bound work (data building, backtests, evaluation) to CPU kernels, which do not consume GPU quota; (2) use the two concurrent GPU commits to keep the queue full; (3) rent a single A100 by the hour on a spot platform (typical market rate roughly $1.0-1.5/hour at the time of writing; the entire Bev project - all iterations plus the final 24-hour run - cost $36 on Prime Intellect). The design keeps every artifact (datasets, adapters, eval suites) in Kaggle-portable form so a rented-GPU run is a drop-in replacement for the training kernel, not a new pipeline.

9. Backtesting and Evaluation Framework

9.1 Validation Methodology

Financial labels overlap in time (one position's outcome window spans many other decision points), so ordinary k-fold cross-validation leaks. The framework uses walk-forward analysis as the outer loop - train 2000-2021, validate 2022-2023, test 2024-2026, then roll the windows forward monthly during operation - with purged k-fold cross-validation and an embargo inside the training window for hyperparameter selection, following Lopez de Prado. Every feature is computed strictly from bars that closed before the decision bar; execution is modeled at the next bar's open plus costs; and the test window is touched exactly once, at the end, by the frozen candidate model. No iteration after test-window results are seen - that rule is written into the roadmap's phase gates.

9.2 Cost Model

> *Planning assumption (to be calibrated against the owner's actual broker): XAUUSD all-in round-trip cost of $0.35 per ounce (typical raw-spread ECN pricing of $0.10-0.30 spread plus commission), slippage of $0.10 per ounce per side on stop orders, and overnight swap applied to positions held past 22:00 UTC. Backtests will be run at 1.0x, 1.5x, and 2.0x this cost assumption; a strategy whose edge dies at 1.5x assumed costs is treated as no edge.*

9.3 Metrics and Gates

Table 8: Core backtest metrics and Phase 4 gate thresholds

Metric	Definition	Phase 4 gate (net of costs, OOS test window)
Annualized Sharpe	Mean daily return / std, annualized by sqrt(252)	Above 1.0 (target 1.5 for paper-trading promotion)
Max drawdown	Peak-to-trough equity decline	Below 20%
Profit factor	Gross profit / gross loss	Above 1.25
Expectancy	Mean trade result in R multiples	Above +0.15R
Trade count	Closed trades in test window	At least 150 (statistical meaningfulness)
Calibration	Brier score and reliability curve on task-level probabilities	Brier below 0.25 on T2 meta-gate; reliability slope 0.8-1.2
Invariance	Choice-order and paraphrase stability (Section 3.4)	100% argmax stability under option permutation; probability shift under 0.02

9.4 Mandatory Baselines

The looped decision model is only credible if it beats, on the same test window and cost model: (1) buy-and-hold gold; (2) an EMA(50/200) trend-following rule; (3) a Donchian(20) breakout rule; (4) an XGBoost classifier on the identical tabular feature set with thresholded entries - the boring-ML control that isolates what the language-model trunk adds; and (5) the unlooped (r=1) version of itself - the ablation that isolates what looping adds. Baselines 4 and 5 are the two that can actually falsify the project's two core bets, and they are cheap: the XGBoost control trains in minutes on CPU kernels, and the r=1 ablation is the same checkpoint family.

9.5 Statistical Robustness

Three further guards against self-deception: deflated Sharpe ratio (correcting for the number of configurations tried - the multiple-testing problem that the automation loop makes worse by making experiments cheap); Monte Carlo bootstrap over trade sequences (5,000 resamples) to put a confidence interval on expectancy and drawdown; and regime slicing (bull/bear/chop years, high/low volatility halves, by-session) so that a headline Sharpe cannot hide a single-regime edge. Results tables will always show the bootstrap interval, never a point estimate alone.

9.6 Tooling

vectorbt (open source, NumPy/Numba-vectorized) runs parameter sweeps and baseline comparisons orders of magnitude faster than event-loop frameworks, which matters inside 12-hour kernels; a small custom event-driven harness executes the composed six-task policy (T1-T6 with the code-level risk wrapper) bar by bar, because the policy's conditional structure (gates, trailing logic, event guards) is not expressible as a pure vectorized signal. Both live in the repo as CPU kernels; neither consumes GPU quota.

10. Proposed System Design: AURUM Decider

10.1 Architecture Overview

AURUM Decider is a 0.45B-class looped decision model. The trunk is Qwen3-0.6B truncated to 20 layers. Looping is applied Huginn-style but adapted for fine-tuning from a pretrained checkpoint: layers 1-12 form the prelude; layers 13-16 form the recurrent core, applied r times; layers 17-20 plus the scoring head form the coda. The packed input sequence is Question, then State, then Options (each with its own position-ID reset), then the answer prompt - the Bev ordering, which lets state tokens read the question first. Position IDs reset at every option boundary (each option's positions start at k+1, independent of its serial position), and the block-causal 4D float mask implements the visibility topology: options attend to question/state and themselves but never to each other; the answer prompt attends to everything. The mask and position IDs are computed once per example and reused identically on every loop iteration - looping mutates hidden states only.

The injection adapter A projects the concatenation of the running state and the prelude output [s; e] from 2048 back to 1024 dimensions before each core pass. Following the risk analysis of Section 4.5, A is initialized so that iteration 1 approximately reproduces the unlooped trunk (identity-dominant initialization), which makes r=1 an exact baseline and lets the loop only improve on it during training. The core is deliberately blind to its iteration index. Training samples r from {1, 2, 3} (geometric weights 0.5/0.33/0.17); inference exposes r as a dial evaluated in {1, 2, 3, 4}.

10.2 Scoring Head and Calibration

The discarded language-modeling head is replaced by a two-layer self-attention scoring head over the extracted vectors: the final token embedding of each option plus the answer-prompt embedding, each augmented with a role embedding (option vs answer) and a task-type embedding (choice/noul/score, and the T1-T6 task identity). A linear projection maps each refined option vector to one scalar logit; unused slots (up to N_max=255) receive negative infinity before softmax. Following the Kev practice, a temperature is fitted post-hoc on a held-out calibration split per checkpoint and shipped with the model; calibration is re-verified monthly during operation (the calibration-drift risk in Section 12).

10.3 Parameter Budget

Table 9: AURUM Decider parameter budget (estimates; to be confirmed in Phase 2)

Component	Parameters	Trainable	Notes
Trunk layers 1-20 (Qwen3-0.6B, truncated)	~314M	No (frozen base)	LoRA rank 16 on last 12 layers adds ~4-5M trainable low-rank weights
Embeddings (tied, vocab 151,936 x 1024)	~155M	No	Input embeddings only; the 2048-wide lm_head projection is discarded
Injection adapter A (2048 to 1024)	~2.1M	Yes	Identity-dominant initialization so r=1 reduces to baseline
Scoring head (2 self-attention layers + roles + task types + linear)	~5-8M	Yes	Includes task-type embeddings for T1-T6
Total	~0.47-0.48B	~12-15M	Trains on a single 16 GB T4 with checkpointing; ~3-7 GPU-hours per full run (estimate)

10.4 Training Recipe

Three stages, each with a go/no-go gate. Stage A (sanity, ~1 GPU-hour): freeze the trunk entirely, train only the head and adapter on a 20,000-example subset; success means the model beats random and class-prior baselines and passes the invariance suite - this isolates head-implementation bugs from everything else. Stage B (main, 3-7 GPU-hours): LoRA on the last 12 layers plus head and adapter, full 120,000-example corpus, cosine schedule, early stopping on the purged validation split; success means matching or beating the XGBoost tabular control on task-level accuracy and clearing the Brier gate. Stage C (optional): unfreeze the full trunk at low learning rate (8-bit optimizer, single T4) only if Stage B plateaus below its gate. The loop is present from Stage B onward; Stage A runs with r fixed at 1 to keep the sanity check minimal.

10.5 Dataset Budget and Mixture

Target corpus of roughly 120,000 examples: about 45% T1/T2 direction and gate tasks sampled from H1 decision points with triple-barrier and meta labels; 20% T3 exit-management tasks sampled from simulated open positions; 15% T4 stop-distance and 10% T5 size tasks; 10% T6 regime checks. Augmentation multiplies the base set: feature-perturbed counterfactual pairs (target 10% of corpus, label-flipped by construction) and 2-3 paraphrase templates per state. Class balance is enforced per task (no-action capped at 50%

of T1 examples; T3 balanced across the four actions). The corpus, its generation code, and its version hash are published as a Kaggle dataset so every training run is exactly reproducible.

11. Implementation Roadmap

The build is sequenced so that every phase produces a measurable artifact, consumes known compute, and ends at a gate that can stop the project cheaply if a core assumption fails. GPU-hour figures are planning estimates against the 30-hour weekly quota.

Table 10: Phased roadmap with artifacts, compute, and success gates

Phase	Deliverables (Kaggle artifacts)	Est. GPU-hrs	Success gate
1. Data pipeline (week 1)	aurum-data-build CPU kernel; HistData ingestion + Dukascopy cross-check; multi-timeframe features; triple-barrier and meta labels; aurum-data-v1 dataset (~120k examples)	0 (CPU)	Zero look-ahead detected by automated audit; label distributions documented; Dukascopy sample comparison within tolerance
2. Baseline decider (weeks 2-3)	aurum-train-baseline kernel (r=1, Stages A/B); eval harness with invariance + calibration suites; XGBoost tabular control	6-12	100% choice-order invariance; task accuracy beats XGBoost control or within 2 points with better calibration (Brier gate); paraphrase flip rate under 10%
3. Looped variant (weeks 3-4)	aurum-train-looped kernel (looped core, r in {1,2,3}); depth sweep at inference r in {1,2,3,4}; A/B vs baseline on identical splits	8-16	Looped beats r=1 baseline on validation + 2022-2023 window on accuracy or Brier; no calibration degradation at operating r
4. Full-policy backtest (weeks 4-5)	aurum-backtest CPU kernel; composed T1-T6 policy with cost model; walk-forward + bootstrap + deflated Sharpe; baselines 1-5	2-4 (inference only)	Section 9.3 gates met on the untouched 2024-2026 test window: Sharpe above 1.0, max DD below 20%, PF above 1.25, 150+ trades, bootstrap CI positive
5. Paper trading (weeks 5-8)	aurum-live-daily scheduled kernel: refresh data, run policy, emit decisions file + risk report; orchestrator reports daily	1-2/week	4+ weeks of live-paper operation with decision latency under 1s, calibration stable (monthly Brier check), no risk-wrapper violations

Phase	Deliverables (Kaggle artifacts)	Est. GPU-hrs	Success gate
6. Extensions (post-MVP)	Portfolio task over added instruments (XAGUSD, FX majors); optional halting head (adaptive r); Qwen3-1.7B trunk upgrade; MT5 bridge for semi-auto execution if the owner opts in	per scope	Defined per extension when Phase 5 evidence is in

First two weeks in concrete terms: Phase 1 lands the data pipeline and publishes aurum-data-v1 with its audit report; Phase 2 Stage A lands the head-sanity run and the invariance suite. By the end of week two the owner reviews three artifacts - the dataset card, the invariance test report, and the Stage A metrics - before any large GPU spend begins. From Phase 2 onward the orchestrator loop of Section 8.4 runs all experiments: the owner never has to touch the Kaggle UI unless a quota decision is needed.

12. Risk Register and Mitigations

Table 11: Risk register (likelihood and impact rated High/Medium/Low)

Risk	L	I	Mitigation
Overfitting to limited gold history	H	H	Purged CV + embargo; single touch of the test window; deflated Sharpe; mandatory baselines incl. XGBoost control; phase gates
Regime dependence (edge is a bull-market artifact)	M	H	Regime-sliced evaluation; walk-forward windows spanning 2022 bear and 2024-2026 bull; regime features in state; monthly refit cadence in operation
Loop adds nothing at 0.45B scale (useful-r collapse)	M	M	r=1 checkpoint is always the deployable fallback; loop is an ablation, not a dependency; success gate framed as beat-or-match
Training instability of looped fine-tune	M	M	Identity-init adapter; sandwich norms; depth-scaled init for new out-projections; Stage A isolates head bugs first
Calibration drift in operation	M	H	Per-checkpoint temperature; monthly Brier checks in paper trading; auto-flag (not auto-trade) on drift
News gaps invalidate bar-based decisions	H	M	Code-level event calendar guards (flat/hold around top-tier releases); no stop-free positions; weekend-flat default

Risk	L	I	Mitigation
Class imbalance (no-trade dominates)	H	M	Per-task balance caps; cost-weighted loss; threshold tuning on the precision-recall curve, gated by trade-count minimum
Kaggle quota exhaustion mid-phase	M	M	CPU kernels for all non-training work; 2 concurrent commits; chunked resumable runs; rented-A100 fallback ($20-40 covers a Bev-scale campaign)
Live data source rate limits or gaps (Yahoo 429s)	M	L	Retry with backoff and headers; HistData monthly re-anchor; Dukascopy secondary; missing-data means no decision (fail-safe flat)
Execution realism gap (backtest flatters live)	M	H	Conservative cost model with 1.5-2x stress runs; next-bar-open execution; slippage on stops; paper trading before any live consideration
Rewording/format fragility (Laya lesson)	M	M	Fixed versioned state format; paraphrase-augmented training; paraphrase invariance in the eval suite
Choice-order leakage into live prompts	L	M	Invariance by construction (mask + position reset) plus 100% permutation test gate every checkpoint

13. Conclusions and Next Steps

The research supports proceeding. Every component of the proposed system rests on a demonstrated precedent: the decision-model interface and its calibration discipline on the Jev ecosystem; the choice-order-invariance machinery and the 20-layer Qwen3-0.6B recipe on the Bev walkthrough; the looped transformer mechanics on Huginn and the 2026 literature; the labeling and validation methodology on the financial machine-learning canon; and the training-and-automation substrate on today's live verification of the owner's Kaggle access. The genuinely novel element - recurrent depth inside a decision model - is novel in the safe direction: the unlooped model is the r=1 special case of its own checkpoint, so the loop can be adopted or dropped on evidence rather than conviction.

The recommended immediate next step is Phase 1: the data pipeline kernel and the aurum-data-v1 dataset, which costs no GPU quota and unblocks everything else. Three decisions from the owner are requested before or during Phase 1: (1) default risk parameters for the sizing task - proposed starting point 0.5% risk per trade with a 2.0% hard cap, owner-adjustable; (2) confirmation of the primary trading horizon - proposed H1 decision cadence with multi-day holds, matching the swing-style profile implied by the research; (3) explicit go-ahead for Phase 1. The owner should also regenerate the Kaggle API token shared in chat

once the project's automation is handed over, as good hygiene for a credential that has passed through a conversation.

14. References

1. TypeSafe AI - Introducing System One Models and Jev (blog), September 2026. https://typesafe.ai/blog/introducing-system-one-models-and-jev
2. TypeSafe AI - homepage: RLCD, latency and cost claims, structured-output error table. https://typesafe.ai
3. JevBench benchmark repository and leaderboard (v1.4.2.2). https://github.com/fstandhartinger/jevbench
4. Kev: Jev-like family of small decision models (code and model cards). https://github.com/jaredpalmer/kev and https://huggingface.co/jaredpalmer/kev-4b
5. Laya: multilingual System 1 decision model (ModernBERT-based), Apache 2.0. https://huggingface.co/convaiinnovations/laya
6. AnyJev (Nokia Applied Research): turning frozen LLMs into decision models. https://github.com/nokia-applied-research/AnyJev
7. Jev's Open Rivals Test the Decision Model Idea - Laya, Kev, AnyJev, DiffusionGemma compared. Trilogy AI Center of Excellence, September 2026. https://trilogyai.substack.com/p/jev-open-decision-models
8. Building a Choice-Order Invariant Decision Language Model from Scratch (Bev Decider 0.4B walkthrough, project-provided document, 2026).
9. Geiping et al. - Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach (Huginn). arXiv 2502.05171. https://arxiv.org/abs/2502.05171
10. Huginn-0125 model card (recurrent-depth 3.5B; depth modification at inference). https://huggingface.co/tomg-group-umd/huginn-0125
11. What Are Looped Transformers? A Technical Deep Dive. saulius.io, September 2026. https://saulius.io/blog/what-are-looped-transformers-recurrent-depth
12. Li, Liu, Zhou, Ma - Transformers are limited to AC0 (constant-depth circuit bounds). ICLR 2024, arXiv 2402.12875.
13. Graves - Adaptive Computation Time for Recurrent Neural Networks, 2016 (per-position halting; Universal Transformer basis). arXiv 1603.08983
14. Azizi et al. - Looped Language Models: A Survey and Meta-Analysis, 2026; Zhang - Recurrent Looped Transformer, 2026; LT2: Linear-Time Looped Transformers, arXiv 2605.20670; Looped Transformers under the Jacobian Lens, arXiv 2609.01924.

15. Qwen3-0.6B model card (28 layers, GQA 16Q/8KV, Apache 2.0). https://huggingface.co/Qwen/Qwen3-0.6B and Qwen3 Technical Report
16. Qwen3.5 hybrid Gated DeltaNet architecture analyses (GDN:full-attention ~3:1). Raschka LLM Architecture Gallery; ai.tekin.cn deep dive, 2026
17. HistData.com - free forex/metals M1 and tick historical data (66 instruments incl. XAUUSD). https://www.histdata.com
18. Dukascopy - free historical data feed (tick-to-monthly CSV export). https://www.dukascopy.com/swiss/english/marketwatch/historical
19. Kaggle Notebooks documentation and community-verified limits (GPU quota, session length, concurrent sessions, storage). https://www.kaggle.com/docs/notebooks
20. vectorbt - vectorized backtesting research engine. https://github.com/polakowo/vectorbt
21. Lopez de Prado - Advances in Financial Machine Learning (triple-barrier labeling, meta-labeling, purged CV, deflated Sharpe). Wiley, 2018.
22. Gold ML literature: Charugulla et al. 2025 (transformer gold futures, PubMed 41082458); arXiv 2512.22606 (LSTM/MLP + Gray Wolf Optimizer); Varela 2025 (FinBERT+LSTM real-time gold framework, ScienceDirect).